Diabetes Prediction System: How It Works

Updated 29 October 2026. Notebook code reviewed: `f2786e3`. This guide describes the audited baseline, its measured results and its unresolved defects. Documentation updates do not imply that the model code has been repaired.

1. Explain the project honestly

The diabetes notebook loads a small tabular benchmark, separates features and labels, creates a stratified train/test split, standardises features using training statistics, fits a linear support-vector classifier and predicts a hard-coded example.

The model logic executes under a documented compatible environment and filename workaround. The published setup does not run as supplied: the notebook requests `diabetes.csv`, but the committed file is named `diabetes.xls`; it is actually CSV text. A second failure occurs with pandas 3.1 because the drop call supplies both `columns` and `axis`.

This is a learning baseline, not an end-to-end validated diagnostic or calibrated risk system. There is no saved diabetes-model bundle, serving entrypoint or model-test suite in the audited code.

> Analogy: this is a working experiment on a laboratory bench with missing setup labels. Reproducing its result matters, but that does not make it a finished instrument ready for every input or environment.

2. Files and execution status

Audited file	Actual role
`Diabetes.ipynb`	One substantive Python cell for the tabular SVM baseline
`diabetes.xls`	A comma-separated benchmark, not an Excel workbook
`project2_capstone.ipynb`	An unrelated plant-image project requiring private Drive data

The audit first executed the unmodified diabetes source and observed the missing-file error. Copying the dataset bytes to a temporary `diabetes.csv` alias exposed the pandas 3.1 error. With pandas 2.2.3 and NumPy 2.2.6, the unchanged cell completed.

Jupyter's TCP and IPC kernel sockets were blocked in the audit workspace. The actual cell source was therefore executed directly as Python. Its training and prediction code ran; a notebook-kernel or browser-UI pass was not claimed. Stored notebook outputs were not used as fresh proof.

The compatible audit used Python 3.12.14, pandas 2.2.3, NumPy 2.2.6, scikit-learn 1.9.1 and SciPy 1.18.1. The repository did not declare this environment; it was established during testing.

3. Understand the input data

There are 768 rows, eight input features and one Outcome label. Class 0 has 500 rows; class 1 has 268. No explicit null cells or exact duplicate rows were found.

Feature	Meaning in this benchmark
Pregnancies	Pregnancy count
Glucose	Glucose measurement
BloodPressure	Diastolic blood-pressure measurement
SkinThickness	Skinfold measurement
Insulin	Insulin measurement
BMI	Body-mass index
DiabetesPedigreeFunction	Benchmark pedigree-related feature
Age	Age

Do not interpret the column list as evidence that the model generalises to every population. The code contains no external validation dataset, prospective evaluation or calibrated probability model.

Hidden missingness

Measurement	Zero-valued records
Glucose	5
BloodPressure	35
SkinThickness	227
Insulin	374
BMI	11

376 rows contain at least one zero in these five fields. The maintained mlbench documentation identifies these zero-coded measurements as missing-data issues. The notebook leaves them untouched. Zero pregnancies should not be treated as the same kind of missing measurement.

> Analogy: a blank thermometer reading entered as zero is still an absent reading, not a very cold observation. A table can contain no NaN cells and still carry missing information through a numeric code.

4. Follow the exact modelling sequence

Split features and outcome

X contains the eight inputs; Y contains Outcome. The problematic current-pandas expression is `drop(columns='Outcome', axis=1)`. The compatible audit did not edit it. A future source repair can use `drop(columns=['Outcome'])` instead, but that change is not implemented by these documents.

Split the rows

The notebook uses 20% test data, stratifies by outcome and fixes `random_state=2`. This gives 614 training and 154 test rows. The test set contains 100 negatives and 54 positives. Stratification preserves approximate label proportions; a fixed seed makes that split reproducible, not universally representative.

Fit preprocessing on training data

The scaler fits on training rows and transforms both partitions. Its fitted means matched training-only means, it saw 614 rows, and the two index sets were disjoint. No scaling leakage was found in this diabetes notebook.

For a feature, standardisation subtracts its training mean and divides by its training standard deviation. Applying a stored transform to a test observation does not refit those statistics. It also does not turn zero-coded missing values into valid measurements.

> Analogy: decide the ruler's units using the training set, then measure test observations with that same ruler. Rebuilding the ruler from the test set would let the exam influence its own preparation.

Fit a linear SVC

The classifier learns a separating hyperplane in the standardised feature space. Support vectors help determine the boundary. A linear kernel was explicitly selected; no hyperparameter search, probability calibration or comparison of challenger models is implemented in the notebook.

Predict the example

The hard-coded eight-value array is scaled using the fitted training scaler, then passed to the classifier. It returns class 0, matching the original stored example. This is a class label, not a personal probability of disease or a validated diagnosis.

5. Reproduce the held-out results

Actual label	Predicted 0	Predicted 1
0	91	9
1	26	28

Metric	Measured result
Training accuracy	78.66%
Test accuracy	77.27%
Majority-class test baseline	64.94%
Balanced accuracy	71.43%
Positive precision	75.68%
Positive recall	51.85%

Metric	Measured result
Positive F1	0.6154
Specificity	91.00%
ROC AUC from decision scores	0.7919

Accuracy is 119 correct labels out of 154. Positive recall is 28 detected positives out of 54; 26 positive records were missed. Reporting only accuracy hides that asymmetry.

The audit's Wilson 95% accuracy interval was 70.04% to 83.18%, under its sampling assumptions. This interval does not validate deployment, calibration or generalisation to another population.

6. Stability checks added by the auditor

These evaluations are audit evidence, not features already present in the notebook. They use the same raw-data scaling-plus-linear-SVC baseline without changing its model or treating zero-coded measurements.

Stratified five-fold result	Mean	Fold standard deviation
Accuracy	77.73%	2.88 percentage points
Balanced accuracy	72.93%	3.27 percentage points
Positive recall	57.05%	5.45 percentage points
ROC AUC	0.8275	0.0282

A Pipeline was fitted separately inside each fold to avoid sharing fitted preprocessing across folds. The standard deviation is not a 95% confidence interval. Ten stratified split replays, seeds 0 through 9, produced accuracy from 68.83% to 79.87%; none was substituted as a more flattering headline score.

7. Input validation is incomplete

Wrong feature counts, non-numeric text, NaN and infinity raised library exceptions. All-zero input and negative glucose, BMI, age or pregnancy counts nevertheless received class-0 predictions. Reversing feature order changed the example to class 1 without a schema-level error.

The code has a positional array rather than named input validation. The original array inference also emits a feature-name warning. The audit demonstrated this behavior; it did not silently add a validation layer.

> Analogy: eight numbers in an envelope are not enough if the labels have fallen off. A correctly shaped vector can still put age in the glucose field. A robust interface must preserve names and meanings as well as length.

8. The unrelated capstone

The other notebook is a plant-image classifier. An execution attempt stopped at missing cv2; directly running its dataset-access cell failed on a private Google Drive path. No replacement images or synthetic training data were created.

Its fit call passes the training generator as validation data. That is a source-confirmed validation flaw in the plant notebook, not evidence of leakage in the separate diabetes split. Its saved plots are not accepted as held-out diabetes results.

9. Study improvements without claiming they exist

Proposed repairs: align filenames; declare dependencies; remove the redundant axis argument; add named feature validation; handle designated zero-coded measurements; save preprocessing and model together; add tests; and report recall/confusion matrices alongside accuracy.

A proposed missing-data design would mark only the designated measurements missing, split the data, then fit an imputer and scaler on training rows only. A proposal might use `SimpleImputer`, `StandardScaler` and `SVC` in a Pipeline. The current notebook does not implement that sequence, and better performance must be measured rather than assumed.

Threshold selection, calibration and external validation would need their own evaluation protocol. Do not tune on the final holdout or imply that increasing recall has no precision/specificity cost.

10. Interview and study checklist

Explain the filename and pandas failures before quoting a metric. Sketch the correct split/fit/transform order. Derive precision and recall from the confusion matrix. Explain why no explicit NaNs does not prove complete data, why a fixed seed is insufficient robustness evidence, and why a decision score is not a calibrated risk probability.

Use the project as a basic SVM learning example with candid limitations. Do not call it a production diagnostic application, an advanced deep-learning system or a model with completed missing-value handling.

Sources: Audited notebook code | SVC reference | StandardScaler reference | Benchmark missing-data notes | Interview questions and tool primer